

Algorithms Problem Solving

Level 3

Fibonacci Series with Recursion



Algorithms Problem Solving

Level 3

Fibonacci Series with Recursion

Definition

Recursive Fibonacci Sequence Generation

-  Program generates Fibonacci terms through recursive function calls.
-  The function passes the current state and remaining count through its parameters.

Explanation

Recursive State Passing

-  Parameters such as ``count``, ``prev1``, and ``prev2`` carry the required Fibonacci state from one call to the next.

-  This avoids recalculating previous Fibonacci values in this particular tail-recursive design.

Next-Term Calculation

-  The next Fibonacci value is calculated from the two previous values.

-  The function then calls itself with updated state values.

Base Condition

-  Recursion stops when the requested number of terms has been generated.

-  A correct base condition is essential to prevent infinite recursive calls.

Procedures

1. Define `generateAndPrintFibonacci(count, prev1, prev2)`.
2. Check the termination condition.
3. Calculate the next Fibonacci number.
4. Print the current term.
5. Call the function recursively with updated values.
6. Start the process from `main()`.

💡 Example

1 1 2 3 5 8 13 21 34 55

➡ Result

The Fibonacci sequence is generated recursively.

Note

- ◆ This implementation is different from the classic recursive formula $F(n) = F(n-1) + F(n-2)$.
- ◆ Passing the previous values as parameters makes the recursion linear rather than branching.
- ◆ Each recursive call creates a stack frame, so auxiliary stack space is $O(N)$.

⊘ Warning

- ◆ The base condition must eventually be reached.
- ◆ Deep recursion can exhaust stack memory.
- ◆ Temporary values must be preserved before updating the previous Fibonacci parameters.



Comparison

▨ Tail-Style Recursive Fibonacci

- ✓ Passes the current state through parameters.
- ✓ Makes one recursive call per generated term.
- ✓ Uses $O(N)$ call-stack space.

▤ Classic Recursive Fibonacci

- ✓ Uses the mathematical definition with two recursive calls.
- ✓ Recomputes overlapping subproblems without memoization.
- ✓ Has exponential time complexity in the naive implementation.

✦✦ Summary

- ✓ Generates Fibonacci values through recursive function calls.
- ✓ Carries previous values through function parameters.
- ✓ Uses a base condition to terminate recursion.
- ✓ Demonstrates a linear recursive approach rather than the expensive branching Fibonacci formula.

Thank you 😊